Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Build a Human-AI Workforce with Autonomous AI Agents

TGS Ref No: TGS-2024043854

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.4

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	12 July 2026	Initial release — Learner Guide covering the autonomous-AI-agent labs (Hermes Agent, OpenClaw, Paperclip).	Dr. Alfred Ang
1.1	14 July 2026	Revised for the 2-day structure and 34 labs (Hermes 14, OpenClaw 12, Paperclip 8) with per-lab reference videos; learning outcomes aligned to the Course Proposal / TSC.	Dr. Alfred Ang
1.2	14 July 2026	Per-lab step numbering; YouTube links removed (videos remain in the labs); Hermes install lab expanded with the official install methods.	Dr. Alfred Ang
1.3	14 July 2026	Deck restyled in the dark masterclass theme end-to-end; added the Hermes masterclass slides (What is Hermes; The Bet: Compounding Value; platform matrix; Top-10 slash commands; Telegram setup; memory L1-vs-L2 and session-search; what-is-a-tool, built-in tool catalog, tools-in-action, cron scheduling, delegation, kanban orchestration and OpenClaw topic slides); Lab 13 rebuilt as the multi-agent video team (profiles + kanban) with detailed Labs 11-14 guides; fade slide transitions and the live Achievements dashboard screenshot.	Dr. Alfred Ang
1.4	15 July 2026	Security masterclass slides added to the Hermes Security lab (defense-in-depth layers; the gateway's six-check default-deny	Dr. Alfred Ang

		trust chain; channel lockdown dials; the three approval modes + the Tirith scanner; a live Dangerous-Command approval; YOLO mode; the unrecoverable-command blocklist floor; prompt-injection scanning of context files). Security lab steps expanded with the channel allowlist and approvals.mode.	

TABLE OF CONTENTS

Introduction

This Learner Guide accompanies the WSQ course Build a Human-AI Workforce with Autonomous AI Agents (TGS-2024043854), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 34 hands-on labs across three autonomous-AI-agent platforms — Hermes Agent, OpenClaw and Paperclip — organised into three topics. Each platform builds on the same skillset: install the runtime, connect models and channels, give the agent memory, extend it with skills and tools, automate it with schedules, secure it with governance, and finally orchestrate multiple agents to deliver a realistic business outcome.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. The labs run on your own laptop and in Docker; where a lab connects to an external service (an LLM provider, a messaging channel, a VPS) use only accounts and credentials you own, and keep API keys and tokens out of prompts and out of version control.

Course Learning Outcomes

- LO1: Analyse AI (LLM-based) applications across a range of industries to identify their capabilities and limitations.
- LO2: Establish the relationship between AI algorithm/agent design and chatbot/agent efficiency.
- LO3: Evaluate and improve the effectiveness of AI (RAG and agent) applications in product development.

Before You Start — Environment Setup

What you need

- A laptop you have administrator rights on — Windows 10/11, macOS 12+ or Ubuntu 22.04+ — with a terminal.
- Node.js 24 LTS (required to run the OpenClaw gateway daemon).
- Docker Desktop (required to self-host Paperclip with Docker Compose, and used for isolated agent sandboxes).
- At least one LLM provider login or API key — Claude (Anthropic), OpenAI, OpenRouter, MiniMax or DeepSeek.
- A free Telegram account for the messaging-channel labs (you will create a bot via BotFather).
- Optional: a VPS (e.g. Hostinger or exe.dev) if you want to run an agent 24/7 beyond the classroom.

Verify your toolchain

Open a terminal and confirm the core prerequisites are present before you begin. Each platform's installer (Hermes, OpenClaw, Paperclip) is run in its own lab; here you only confirm the building blocks.

```
$ node --version     # Node.js 24.x for OpenClaw

$ docker --version   # Docker Desktop for Paperclip and sandboxes

$ docker compose version

$ git --version      # to clone the course labs repository
```

Conventions used in every lab

- Commands are run from your terminal; a leading sudo is used only where elevated rights are required.
- Placeholders such as <API_KEY>, <BOT_TOKEN> and <HOST> are replaced with your own values.
- Store provider keys and channel tokens in each platform's config (never in prompts or in git).
- Keep approval prompts, budgets and sandboxing on so agents act under human oversight.
- Each capstone orchestrates multiple agents — start the single-agent labs first so the pieces are familiar.

Topic 01 — Hermes Agent — Your Autonomous Personal Chief-of-Staff (33%)

Install & Setup · Deployment · Memory & Plugins · Skills · Providers & Model · MCP & Tools · Crons · Subagents · Profile & Kanban · Security

Key concepts

- Hermes Agent (Nous Research) installs on your own machine — via the install script or the Hermes Desktop app — and requires a model with a >=64,000-token context window.
- Memory and plugins make the agent stateful and extensible: a three-layer memory (agent-curated notes, FTS5 cross-session recall, Honcho user modelling) plus installable skills and MCP tools.
- Providers & models are swappable with no lock-in (Nous Portal, OpenRouter, OpenAI, any endpoint); MCP servers and the Tool Gateway (web search, image, TTS) give the agent real-world reach.
- Cron jobs automate recurring work; subagents & delegation split work across isolated backends; the profile and Kanban board let you supervise the agent's tasks — all under security controls.

Lab 1 — Install and Setup Hermes

Learning outcome: LO1: Install and configure Hermes Agent on a local machine..

Goal: Install Hermes Agent on your laptop using any of the official install methods (Desktop installer, install script, PowerShell, from source, or Nix), run the first-time setup wizard, and confirm a healthy install. This is the foundation for the Athena chief-of-staff agent you build across the track. Prerequisite: Git; the installer auto-handles Python 3.11, Node.js v22, ripgrep and ffmpeg.

What you'll build

A working Hermes Agent install that passes 'hermes doctor' and opens the TUI. (Tools: Hermes Agent, Nous Portal, terminal.)

Step-by-step

1. Choose your installation method Hermes offers several official install paths — pick the ONE that matches your machine: > If you install via the Desktop app (Option A), you can skip Steps 2–3. If you install via the CLI first, you can add the desktop app later with hermes desktop.

2. Option B — install script (Linux / macOS / WSL2 / Termux) Run the official one-line installer. It downloads the Hermes runtime, places it under your user directory, and wires up the hermes command:

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

3. Option C — Windows PowerShell (native Windows) On native Windows (no WSL2), run the PowerShell installer instead:

```
iex (irm https://hermes-agent.nousresearch.com/install.ps1)
```

4. Install Hermes Desktop (macOS / Windows) Hermes Desktop gives you a GUI over the same agent. Either download the installer from <https://hermes-agent.nousresearch.com/> and run it, or — if you already installed the CLI in Step 2/3 — launch it directly:

```
hermes desktop
```

5. Reload your shell so the hermes command is on PATH The installer adds Hermes to your PATH via your shell profile, but your current terminal session hasn't re-read that file yet. Reload it:

```
source ~/.zshrc
```

6. Run the first-time setup wizard Launch the interactive setup wizard and connect Hermes to your Nous Portal account. This is where you sign in, pick defaults, and let Hermes provision a model provider:

```
hermes setup --portal
```

7. Check the install is healthy Run the built-in diagnostic. It checks the binary, config, PATH, provider connectivity, and the local runtime:

```
hermes doctor
```

8. Launch the agent and say hello Open the terminal UI (TUI) and start your first conversation with the agent — this is Athena's very first session:

```
hermes --tui
```

Test it

'hermes doctor' reports all green and the TUI opens and replies to a message.

Note: Full commands and screenshots are in labs/lab-01-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 2 — Deployment — Local Install & Hermes Desktop App

Learning outcome: LO1: Deploy Hermes locally and run it through the Hermes Desktop app..

Goal: Run Hermes as a persistent local deployment (preferred) and also install the Hermes Desktop app for a GUI experience. Confirm the local gateway is serving and keep the runtime up to date.

What you'll build

A local Hermes deployment plus the Desktop app, both talking to the same agent. (Tools: Hermes Agent, Hermes Desktop, terminal.)

Step-by-step

1. Watch the reference videos Watch both videos above. The primary shows the local deployment; the additional walkthrough covers the Desktop app in more depth. Note any UI screens that differ from the steps below.

2. Confirm the local install runs (preferred deployment) Verify the CLI runtime you installed in Lab 1 is present and check its version. The local install is the preferred deployment — it keeps the agent runtime on your machine:

```
hermes --version
```

3. Download and install the Hermes Desktop app Download the Desktop app for your OS from the official site and install it like any normal application:

4. Open the Desktop app and sign in Launch the Hermes Desktop app and sign in with the same Nous Portal account you used for the CLI in Lab 1. Signing in with the same account is what makes the Desktop app and the CLI drive the same agent. > This step is UI-only — there is no terminal command. In the app, choose Sign in with Nous Portal and complete the browser/authentication prompt.

5. Verify the local gateway is serving The gateway is the local service that both the CLI and Desktop app connect to. Confirm it is up and healthy:

```
hermes gateway status
```

6. Keep the runtime up to date Update Hermes to the latest release so the CLI and Desktop app stay compatible:

```
hermes update
```

Test it

Both the local CLI and the Desktop app drive the same agent; 'hermes gateway status' is healthy.

Note: Full commands and screenshots are in labs/lab-02-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 3 — Memory and Plugins

Learning outcome: LO2: Give the agent persistent memory and extend it with plugins..

Goal: Explore Hermes' three-layer memory (agent-curated notes, FTS5 cross-session recall, Honcho user modelling). Teach Athena a preference, recall it in a fresh session, and enable a plugin to extend behaviour.

What you'll build

Athena remembers a stated preference across sessions and runs an enabled plugin. (Tools: Hermes Agent, memory, plugins.)

Step-by-step

1. Teach Athena a durable preference Start a session (hermes --tui) and give Athena a clear, memorable instruction that should persist. Explicitly asking it to remember nudges the agent to write the fact into its curated-notes memory layer:

```
Remember I prefer concise, bulleted summaries.
```

2. Start a new session and confirm the preference is recalled Resume the conversation in a fresh session. The --continue flag brings back prior context so you can test cross-session recall:

```
hermes --continue
```

3. Inspect stored memory / sessions List the stored sessions so you can see the memory Hermes is persisting behind the scenes:

```
hermes sessions list
```

4. Enable a plugin to extend the agent Enable a plugin to add capability beyond the base agent. Plugins are toggled from the Hermes plugins UI or config:

Test it

A preference taught in one session is recalled in a new session, and the enabled plugin is active.

Note: Full commands and screenshots are in labs/lab-03-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 4 — Skills

Learning outcome: LO3: Extend the agent with installable, self-improving skills..

Goal: Browse and install open-standard skills (agentskills.io / Skills Hub) so Athena can perform new tasks. Skills are portable and can self-improve as the agent uses them.

What you'll build

Athena equipped with at least one installed skill that it can invoke on request. (Tools: Hermes Agent, agentskills.io, Skills Hub.)

Step-by-step

1. Browse the skills hub Open the skills catalogue to see what's available to install:

```
hermes skills browse
```

2. Search for a skill by keyword Narrow the catalogue to something useful for a chief-of-staff agent — for example calendar, email, research, or pdf:

```
hermes skills search <keyword>
```

3. Install a skill Install a skill by its full identifier (copy it from the search results). The pattern is owner/skills/name:

```
hermes skills install <owner/skills/name>
```

4. Ask the agent to use the new skill and observe self-improvement Open the TUI and ask Athena to perform a task that the new skill enables. Watch it select and run the skill: > This step is conversational — no fixed command. Phrase a request that maps to the skill (e.g. for a summarize skill: "Summarize this article for me: <paste text>"). As the agent uses the skill repeatedly, it refines how it invokes it — that's the self-improvement aspect.

Test it

The installed skill appears in the agent's toolset and runs when invoked.

Note: Full commands and screenshots are in labs/lab-04-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 5 — Providers & Model

Learning outcome: LO2: Connect an LLM provider and switch models with no lock-in..

Goal: Connect a model provider (Nous Portal, OpenRouter, OpenAI or any endpoint) and select a model that meets the >=64,000-token context requirement. Switch providers to compare speed and quality.

What you'll build

Athena running on a chosen provider/model, with the ability to switch on demand. (Tools: Hermes Agent, Nous Portal, OpenRouter.)

Step-by-step

1. Open the interactive provider/model picker Launch the interactive picker to see available providers and models and select one:

```
hermes model
```

2. Set a model explicitly Instead of (or in addition to) the picker, set the active model directly in config:

```
hermes config set model anthropic/claude-opus-4.6
```

3. Add a provider key if using a cloud endpoint If your chosen model runs on a cloud endpoint (e.g. OpenRouter), store the API key in config so Hermes can authenticate:

```
hermes config set OPENROUTER_API_KEY sk-or-...
```

4. Confirm the active model meets the ≥64k context rule Reopen the picker to confirm the active model and its context window:

```
hermes model
```

Test it

The agent starts on the selected model and can be switched to another provider without reinstalling.

Note: Full commands and screenshots are in labs/lab-05-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 6 — MCP and Tools

Learning outcome: LO3: Give the agent real-world reach with MCP servers and the Tool Gateway..

Goal: Add Model Context Protocol (MCP) servers in the Hermes config and use the bundled Tool Gateway (web search, image generation, TTS) so Athena can act on the outside world.

What you'll build

Athena wired to an MCP server (e.g. GitHub) and able to use a Tool Gateway tool. (Tools: Hermes Agent, MCP, Tool Gateway.)

Step-by-step

1. Open the Hermes config to add an MCP server Edit your Hermes config file and locate (or create) the mcp_servers: section:
2. Add a GitHub MCP server entry Add an entry that launches the official GitHub MCP server via npx. A typical block looks like this:

```
mcp_servers:

  github:

    command: npx

    args: ["-y", "@modelcontextprotocol/server-github"]

    env:

      GITHUB_PERSONAL_ACCESS_TOKEN: "ghp_your_token_here"
```

3. Restart so the MCP server is picked up, then verify Restart Hermes (close and reopen the TUI, or restart the gateway) so it reads the new config, then run the diagnostic:

```
hermes doctor
```

4. Use a Tool Gateway tool from a chat Open the TUI and ask Athena to use a bundled Tool Gateway capability — web search, image generation, or text-to-speech: > This step is conversational. For example: "Search the web for today's top AI agent news and list 3 headlines," or "Generate an image of a friendly robot assistant." Confirm the agent invokes the tool and returns the result.

Test it

The agent lists the new MCP server's tools and completes a task using a Tool Gateway tool.

Note: Full commands and screenshots are in labs/lab-06-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 7 — Cron Jobs & Automation

Learning outcome: LO4: Automate recurring agent work on a schedule..

Goal: Schedule Athena to run recurring jobs — for example an 8am daily briefing sent to your messaging app — and verify the scheduled run fires and produces output.

What you'll build

A scheduled job (e.g. a daily briefing) that runs automatically and delivers a result. (Tools: Hermes Agent, scheduler/crons.)

Step-by-step

1. Define a recurring job (daily 8am briefing) Create a recurring automation in the Hermes automations/crons section — for example a daily 8am briefing:
2. Confirm the automation is registered Run the diagnostic to confirm Hermes has picked up the new scheduled job:

```
hermes doctor
```

3. Trigger the job once to confirm it produces output Manually run the job once (rather than waiting for 8am) to confirm it produces the expected briefing. > This is typically a "run now" action in the automations UI, or a trigger command shown in the docs. Confirm the briefing is generated and delivered to your chosen destination.
4. Confirm the next scheduled run is queued Check that after the manual run, the next run is still scheduled for its normal time (08:00 tomorrow), so the automation keeps recurring.

Test it

The scheduled job runs at its time (or on manual trigger) and delivers the expected briefing.

Note: Full commands and screenshots are in labs/lab-07-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 8 — Subagents & Delegation

Learning outcome: LO5: Delegate work to isolated subagents across backends..

Goal: Turn Athena into a coordinator that delegates tasks to worker subagents running on isolated terminal backends (local, docker, ssh, modal), then aggregates their results.

What you'll build

A coordinator agent that delegates a task to one or more worker subagents and combines the output. (Tools: Hermes Agent, terminal backends (docker/ssh/modal).)

Step-by-step

1. Choose an isolated backend for worker agents Set the terminal backend that worker subagents will run on. docker gives each worker an isolated container:

```
hermes config set terminal.backend docker
```

2. Ask the coordinator to delegate a multi-part task Open the TUI and give Athena (the coordinator) a task with clearly separable parts so it delegates each to a worker subagent: > Conversational step. For example: "Delegate this: subagent A researches competitor pricing, subagent B drafts a comparison table, subagent C writes a one-paragraph recommendation. Then combine their work." Watch the coordinator spin up workers.

3. Observe each subagent run in isolation and report back Watch each worker subagent execute on the isolated backend and return its portion of the result. Confirm they run separately (each in its own container/environment when using docker) rather than in the coordinator's session.

4. Review the coordinator's aggregated result Confirm the coordinator combines the workers' outputs into one coherent deliverable — the research, the table, and the recommendation merged into a single response.

Test it

The coordinator delegates to worker subagents on an isolated backend and returns a combined result.

Note: Full commands and screenshots are in labs/lab-08-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 9 — Profile and Kanban

Learning outcome: LO5: Supervise the agent through its profile and Kanban board..

Goal: Configure the agent's profile and use the Hermes Kanban board to watch tasks move through todo -> in progress -> done, giving you human oversight of what Athena is working on.

What you'll build

A configured agent profile and a Kanban board tracking the agent's live tasks. (Tools: Hermes Agent, profile, Kanban board.)

Step-by-step

1. Set up the agent profile (identity, defaults, preferences) Configure Athena's profile — its identity, default model/backend, and behavioural preferences. > Open the profile settings in the Desktop app (or the profile section of ~/.hermes/config.yaml). Set the name to Athena, role to personal chief of staff, and default preferences (concise bulleted output, timezone, working hours). The exact profile fields are version-specific — verify in the video / Hermes docs (https://hermes-agent.nousresearch.com/docs/).

2. Open the Kanban board view Open the Kanban board that visualizes the agent's tasks. > In the Desktop app, open the Kanban / Tasks view. You should see columns such as Todo, In Progress, and Done.

3. Assign a task and watch it move todo → in progress → done Give Athena a task and watch the corresponding card move across the board: > Assign a task (e.g. "Draft my weekly team update"). A card appears in Todo, moves to In Progress as the agent works, and lands in Done when finished. This is your live oversight of what the agent is doing.

4. Use the board to review and accept completed work When a card reaches Done, open it, review the agent's output, and accept (or send it back with feedback). This human-in-the-loop review is the point of the board.

Test it

The profile is set and the Kanban board shows tasks progressing across its columns.

Note: Full commands and screenshots are in labs/lab-09-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 10 — Security

Learning outcome: LO4: Secure the agent with isolation, secrets and approvals..

Goal: Harden Athena with defense in depth: restrict who can message the agent (default deny), run risky work in an isolated terminal backend, manage secrets safely, and require approval before sensitive actions — applying least privilege throughout.

What you'll build

A hardened agent that allowlists its users, sandboxes execution, protects secrets and prompts for approval on risky actions. (Tools: Hermes Agent, gateway allowlists, terminal backends, secrets management.)

Step-by-step

1. Restrict who can message the agent — set the channel allowlist (default deny) Decide who is allowed to talk to Athena. The gateway authorizes every inbound message with a six-check chain — per-platform allow-all, DM pairing, the platform allowlist, the global allowlist, global allow-all, then default deny. Nothing configured means everyone is denied (fails closed), with a startup warning telling you exactly that. In the dashboard open Channels → Configure for your channel (e.g. Telegram) and set the allowlist to your own user ID: - Allowed user IDs (TELEGRAM_ALLOWED_USERS) — comma-separated IDs that may use the bot (get yours from @userinfobot). - Allow all users (TELEGRAM_ALLOW_ALL_USERS) — leave off; it opens the bot to anyone (dev

only). - Unknown DMs can also be admitted one-by-one via pairing codes you approve.

2. Isolate execution in a sandboxed backend Set the terminal backend to a sandboxed container so risky commands never run directly on your host:

```
hermes config set terminal.backend docker
```

3. Store provider/tool secrets via config rather than plain text Store API keys through the Hermes config mechanism instead of pasting them into prompts or scripts:

```
hermes config set <PROVIDER>_API_KEY <value>
```

4. Require approval before the agent runs risky actions — set the approval mode Set the approval mode so the agent pauses and asks you before executing sensitive actions (shell commands, file deletions, external posts):

```
hermes config set approvals.mode manual
```

5. Apply least privilege — grant only the tools/skills the task needs Review the tools, skills, and MCP servers Athena has access to and disable anything not needed for the current work. > Conversational/config step: trim the enabled skills (Lab 4) and MCP servers (Lab 6) to the minimum. Least privilege means the agent can only do what the task actually requires.

Test it

Only allowlisted users reach the agent; risky actions run in the sandbox and only after approval; secrets are not stored in plain text.

Note: Full commands and screenshots are in labs/lab-10-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 11 — Use Case — Make a Video with Hyperframe

Learning outcome: LO5: Apply the agent to a real creative use case — produce a video with Hyperframe..

Goal: Put Athena to work on an end-to-end creative task: use the agent to drive the Hyperframe tool and generate a short video from a concept brief, then review and refine the result.

What you'll build

A short video generated end-to-end by the agent using Hyperframe. (Tools: Hermes Agent, Hyperframe.)

Step-by-step

1. Install / verify the Hyperframe video skill Wire Hyperframe into Athena the same way you added skills in Lab 4. In a terminal, search the skills hub for the Hyperframe

(video) skill and install the match; if your build integrates Hyperframe as an MCP server or Tool Gateway tool instead, add it the way you added tools in Lab 6.

```
hermes skills search hyperframe
```

```
hermes skills install <owner/skills/hyperframe>
```

2. Set the Hyperframe credential via config If Hyperframe needs an API key or account token, store it through Hermes config so it never sits in plain text. Get the key from your Hyperframe account page, then set it and confirm no error is printed.

```
hermes config set HYPERFRAME_API_KEY <your-key>
```

3. Confirm the tool is loaded, then start a session Run the health check and confirm it reports green with the new skill/tool listed. Then open a chat session (TUI or Desktop app) and ask Athena "What video tools do you have?" — it should name Hyperframe in its reply. If it doesn't, restart Hermes so the new tool is picked up.

```
hermes doctor
```

```
hermes --tui
```

4. Brief the agent with a concrete 3-sentence brief Type a tight, three-sentence creative brief covering topic, style/tone, and length + extras. A concrete brief is the single biggest lever on output quality — vague briefs produce generic videos. For example: > "Make a 30-second explainer video introducing 'Athena, my AI chief of staff'. Clean, modern visual style with an upbeat tone. Keep it to 30 seconds and add on-screen captions."

5. Have the agent generate the video and wait for the render Ask Athena to generate the video now. You should see the agent invoke the Hyperframe tool in the session transcript (a tool-call entry), then wait — rendering typically takes a few minutes for a 30-second clip. When it finishes, Athena reports back with a link or file path to the rendered video; if the tool call errors, check the credential from step 3.

6. Play the result, give one round of feedback, and regenerate Open and play the video end-to-end. Then give Athena one specific round of feedback — change exactly the things that bothered you, e.g. "Slow the pacing, make the captions larger, and warm up the colour palette" — and ask it to regenerate. Compare the two versions; you should see your feedback reflected in version 2.

7. Export / save the final video and note where the file lands Ask Athena to save/export the final cut and tell you the exact file path. Outputs typically land in the session's workspace under your Hermes home directory — open it and confirm the file plays from disk:

```
open ~/.hermes/
```

Test it

The agent produces a playable video from your brief using Hyperframe, and you can locate the saved file.

Note: Full commands and screenshots are in labs/lab-11-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 12 — Visualization

Learning outcome: LO5: Visualize agent activity and workflow data..

Goal: Use Hermes' visualization capability to see the agent's workflow, task flow and data at a glance, making the agent's behaviour transparent and easier to supervise.

What you'll build

A visualization of the agent's workflow / activity you can read and interpret. (Tools: Hermes Agent, visualization.)

Step-by-step

1. Run a real multi-step session so there is something to visualize A visualization of an empty agent is empty. Start a fresh session and give Athena a task that forces several tool calls in sequence — for example: > "Research the top 3 AI agent platforms, compare their pricing, and summarize in a table." Let it run to completion; the web searches, reads, and summarization steps it performs become the nodes of your graph.

2. List sessions and note the ID of the one to visualize In a terminal, list your recent sessions and copy the ID (or title/timestamp) of the session you just ran. You will visualize this specific session rather than "everything", which keeps the diagram readable.

```
hermes sessions list
```

3. Open the visualization view In the Desktop app, open the Visualization / Activity view from the sidebar and select the session from step 3. You should see the session's activity surface — task flow, tool calls, and timing. > The exact menu location and view name are version-specific — verify in the video / Hermes docs (https://hermes-agent.nousresearch.com/docs/).

4. Generate a workflow graph of the session Ask Athena (or use the view's generate button) to render the session as a workflow diagram: > "Visualize the steps you took in this session as a workflow diagram." A graph should render in which each node is a tool call (web search, file read, summarize…) and edges show the order the calls ran in. If nothing renders, confirm you selected a session that actually used tools.

5. Read the graph — nodes, sequence, and time Walk the diagram from start to finish and name what each node did: which tool, with what input, producing what output. Check the sequence matches what you asked for, and look for where time was spent (long-running nodes) or where the agent looped/retried. This is the supervision skill: confirming from the graph that the agent did what you expected — nothing more, nothing less.

6. Export or screenshot the visualization Save the diagram for your records — use the view's export button if your build has one, otherwise take a screenshot. Keep it with the lab evidence; you will reuse this technique whenever you need to audit an agent run.

7. Use the graph to explain what the agent did Turn to a classmate (or write three sentences) and explain the run using only the graph: what the goal was, which tools ran in what order, and where the result came from. If you can narrate the run from the diagram alone, the visualization has done its job of making Athena's behaviour transparent.

Test it

A workflow visualization of a real session renders; you can name each node (tool call) and explain what the agent did.

Note: Full commands and screenshots are in labs/lab-12-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 13 — Build a Multi-Agent Video Team (Profiles + Kanban)

Learning outcome: LO3: Orchestrate a team of profiles through the Kanban board to produce a video..

Goal: Build a team of three Hermes profiles — a researcher (long-context model) that reads sources and writes the video brief, a scriptwriter (cheap model) that turns the brief into a narrated script, and a video_producer (code/tool model) that renders the video with TTS and slides — then create a Kanban board task, decompose it into child tasks, and let the orchestrator route each child task to the right profile by its description. The board is a durable SQLite database (~/.hermes/kanban.db) and each task gets its own workspace.

What you'll build

A three-profile video team whose Kanban tasks flow triage -> todo -> ready -> running -> done and deliver a rendered video. (Tools: Hermes Agent, profiles, Kanban board, dashboard.)

Step-by-step

1. Design the team on paper first Before creating anything, write down the three roles, the kind of model each needs, and — most importantly — the one-line description the orchestrator will route by. The description is the routing contract: it must say exactly what the profile takes in and hands off. Note the pipeline shape: brief → script → video. Each profile's output is the next profile's input.

2. Create the researcher profile Create the first profile on a long-context model (it will read sources whole). The --description text is what the orchestrator matches child tasks against, so type it exactly as designed:

```
hermes profile create researcher --description "Reads sources, writes the video brief + key points"
```

3. Create the scriptwriter profile Create the second profile on a cheap model — turning a brief into a script is high-volume, low-difficulty work, so this is where you save money:

```
hermes profile create scriptwriter --description "Turns the brief into a narrated script + scene list"
```

4. Create the video_producer profile Create the third profile on a code/tool-capable model — it must drive TTS and slide-rendering tools reliably:

```
hermes profile create video_producer --description "Turns the script into a rendered video with TTS + slides"
```

5. Verify the team on the Profiles page Open the dashboard and go to the /profiles page in the sidebar. You should see all three profiles listed, each showing its model and its description. Read each description back critically — a vague description here causes wrong routing later. Fix any typos now (edit the profile or recreate it).

6. Create the Kanban board task Create the top-level task on the board and assign the first stage to the researcher. Pick a real topic you care about and keep the deliverable measurable ("60-second explainer video"):

```
hermes kanban create "Produce a 60-second explainer video on <topic>" --assignee researcher
```

7. Decompose the task into child tasks Ask Hermes to break the top-level task into child tasks:

```
hermes kanban decompose <id>
```

8. Watch the lanes as the team works Open the dashboard /kanban page and watch the board dispatch. Every task moves through the lifecycle triage → todo → ready → running → blocked → done → archived. The research child should enter running first (it runs on the researcher profile's model), then hand off so the script task becomes ready, and so on down the pipeline. A task sitting in blocked is asking for a human — open it and read why.

9. Review each task's workspace deliverables Click into each completed child task. Every task gets its own workspace (a scratch directory / dedicated dir / git worktree, depending on configuration) where its outputs live. Open the researcher task's workspace and read the brief; open the scriptwriter's and read the script + scene list; open the video_producer's and play the rendered video. Confirm each stage's output actually fed the next — the script should quote the brief's key points, and the video should follow the scene list.

10. Accept the final video to done If the video meets the brief, accept the top-level task so it moves to done (and later archived). If it doesn't, add a comment on the relevant child task with concrete feedback (e.g. "narration too fast in scene 2") and send that task back through the board rather than redoing everything — that is the point of decomposed work.

11. Reflect: why this beats one big agent Look back at the run: the expensive long-context model only did research, the cheap model wrote the script, and the tool model rendered. One prompt to a single agent would have used the expensive model for everything. Note one sentence on cost and one on reviewability (you could inspect and reject per stage).

Test it

Three profiles exist, the decomposed Kanban tasks route to the right profile by description, and the accepted board delivers a rendered video.

Note: Full commands and screenshots are in labs/lab-13-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 14 — Webhook

Learning outcome: LO3: Integrate the agent with external systems via webhooks..

Goal: Wire the agent to the outside world with webhooks: trigger the agent from an incoming webhook payload and/or have it call an outgoing webhook, then secure the endpoint.

What you'll build

A webhook that triggers the agent (or that the agent calls), verified end-to-end. (Tools: Hermes Agent, webhooks.)

Step-by-step

1. Confirm the local gateway is serving Webhooks ride on the local gateway — if it isn't up, nothing can receive a request. Check its status and note the host/port it reports (typically localhost plus a port); that host/port is the base of your webhook URL.

```
hermes gateway status
```

2. Create / enable a webhook endpoint on the dashboard Open the dashboard and click Webhooks (/webhooks) in the sidebar. Click New webhook (or the enable toggle), give it a recognizable name like briefing-trigger, and save. The dashboard generates a unique endpoint URL for it. > The exact page layout and creation flow are version-specific — verify in the video / Hermes docs (https://hermes-agent.nousresearch.com/docs/).

3. Copy the endpoint URL and map it to an agent action Use the copy button next to the new webhook to copy its full URL, and paste it somewhere handy. Then set what the webhook does: map incoming payloads to an agent action — for this lab, have it run a short briefing (e.g. instruction: "When this webhook fires, read the payload's message field and act on it"). Without a mapping, a payload is accepted but nothing happens.

4. Send a test payload with curl From a terminal, simulate an external system by POSTing a JSON payload to the endpoint. Replace <endpoint-url> with the URL you copied in step 4:

```
curl -X POST <endpoint-url> \

  -H "Content-Type: application/json" \

  -d '{"event":"test","message":"Trigger Athena briefing"}'
```

5. Watch the agent react Switch to the chat/session view (or the logs) and confirm the payload actually triggered Athena: the mapped action runs and its output appears — e.g. a briefing message referencing "Trigger Athena briefing" from your payload. Also check the webhook's row on the /webhooks page; most builds show a delivery/last-triggered indicator. Accepted-but-no-action means the mapping in step 4 isn't wired — fix it and re-send.

6. Secure the endpoint, then re-send with the secret An open webhook lets anyone on the network drive your agent, so lock it down. On the webhook's settings, add a shared secret/token (and an IP allowlist if your build offers one), save, and re-send the test including the secret header — it should still return 2xx and trigger the agent:

```
curl -X POST <endpoint-url> \

  -H "Authorization: Bearer <secret>" \

  -H "Content-Type: application/json" \

  -d '{"event":"test","message":"Trigger Athena briefing"}'
```

7. Confirm unauthorized calls are rejected Re-run the step 5 curl (the one without the secret header). It must now be rejected with 401/403, and Athena must not act. If it still returns 2xx, the secret isn't enforced — re-check the webhook's security settings and save again. You now have a working, secured inbound integration.

Test it

A test payload to the webhook triggers the agent's mapped action, and calls without the secret are rejected.

Note: Full commands and screenshots are in labs/lab-14-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Topic 02 — OpenClaw — Automating a Business Back-Office (33%)

Install · Providers · Channels · Skills · Tools · Commands · Crons · Memory · Dreaming · Security · Multi-Agent · Use Cases

Key concepts

- OpenClaw runs on Node.js 24 LTS as a long-running gateway daemon that hosts every channel, tool, cron and skill for the Nimbus Supplies back office.
- Providers are connected by OAuth sign-in (OpenAI Codex, MiniMax) or API key (Anthropic, DeepSeek, OpenRouter); skills from skills.sh and integrations such as Firecrawl and AgentMail extend the agent.
- 'Dreaming' lets the agent reflect during idle time — reviewing memory and generating follow-ups — while profiles and allow/deny lists keep every channel least-privileged and audited.
- Crons schedule recurring work, a heartbeat auto-restarts the gateway, and a multi-agent capstone splits work across cooperating Sales, Research and Ops agents for real back-office use cases.

Lab 15 — Install OpenClaw

Learning outcome: LO1: Install and configure an autonomous AI agent platform locally and on a container/VPS..

Goal: The learner installs the OpenClaw platform on their own machine (or a cloud VPS) by first installing Node.js 24 LTS, then installing OpenClaw via npm and running the onboarding wizard, and verifies the install with doctor and gateway status. This gateway becomes the back-office brain of the Nimbus Supplies running example.

What you'll build

A running OpenClaw gateway daemon (local or VPS) verified with openclaw doctor (Tools: OpenClaw, Node.js 24 LTS, npm, openclaw gateway.)

Step-by-step

1. Install Node.js 24 LTS (24 recommended; 22.16+ minimum). OpenClaw runs on Node. Windows — download the Node 24 LTS installer from <https://nodejs.org/> and run the .msi (WSL2 is recommended for stability), then check:

```
node -v
```

```
npm -v
```

2. Install OpenClaw — Option A (npm, recommended, cross-platform). After Node is installed:

```
npm install -g openclaw@latest
```

```
openclaw onboard
```

3. Install OpenClaw — Option B (installer script, fallback if npm fails). macOS / Linux / WSL2:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

4. Install OpenClaw — Option C (from source, advanced). Requires git and pnpm:

```
git clone https://github.com/openclaw/openclaw.git

cd openclaw

pnpm install && pnpm build && pnpm ui:build

pnpm link --global

openclaw onboard --install-daemon
```

5. (Optional) Install on a VPS so Nimbus Supplies runs 24/7. The same three-step flow works on any Ubuntu host — a Hostinger KVM VPS or an exe.dev sandbox. SSH in first (ssh root@<your-vps-ip>), then run:

```
# Step 1 — Node.js 24

curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -

sudo apt-get install -y nodejs

node -v && npm -v


# Step 2 — OpenClaw (note: sudo for a global install on a fresh VPS)

sudo npm install -g openclaw@latest


# Step 3 — onboarding wizard

openclaw onboard
```

6. Verify the installation.

```
openclaw --version

openclaw doctor

openclaw gateway status
```

7. Confirm the gateway is running (and set to auto-start). The gateway is the long-running daemon that will host every channel, tool, and cron for Nimbus Supplies:

```
openclaw gateway status

openclaw gateway start   # if it is not already running
```

Test it

openclaw --version prints a version, openclaw doctor shows green checks for Node 24 and the gateway daemon (provider/channel checks pending until Labs 16-17), and openclaw gateway status reports running.

Note: Full commands and screenshots are in labs/lab-15-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 16 — Models & Providers

Learning outcome: LO2: Connect an LLM model/provider to an agent via OAuth or API key..

Goal: The learner connects a language model to OpenClaw, trying the supported provider options — OAuth sign-ins (OpenAI Codex, MiniMax) and API-key providers (Anthropic, DeepSeek, OpenRouter) — learns where credentials are stored under ~/.openclaw/auth/, hot-swaps between models, and runs a Nimbus Supplies smoke test.

What you'll build

A Nimbus Supplies agent wired to a working model provider that passes openclaw model test (Tools: OpenClaw, OpenAI Codex / MiniMax (OAuth), Anthropic / DeepSeek / OpenRouter (API key).)

Step-by-step

1. See what is available and what is currently selected.

```
openclaw models list
```

```
openclaw model current
```

2. Option A — OpenAI Codex (OAuth, GPT-5.5). OAuth uses the openai-codex provider so you sign in with your ChatGPT Plus / Pro subscription instead of paying per token. Credentials are stored in ~/.openclaw/auth/ (profile at ~/.openclaw/auth-profiles/openai-codex.json); refresh is automatic.

```
openclaw models auth login --provider openai-codex
```

3. Option B — MiniMax (OAuth, MiniMax-M2.7). Enable the OAuth plugin, restart the gateway, then log in and set it as default:

```
openclaw plugins enable minimax-portal-auth
```

```
openclaw gateway restart
```

```
openclaw models auth login --provider minimax-portal --set-default
```

4. Option C — Anthropic (API key, Claude Opus 4.7). Get a key from <https://console.anthropic.com/settings/keys>:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
openclaw config set provider anthropic
```

```
openclaw config set model anthropic/claude-opus-4-7
```

```
openclaw model test
```

5. Option D — DeepSeek (API key, V4). Get a key from <https://platform.deepseek.com/api_keys>:

```
export DEEPSEEK_API_KEY="sk-..."
```

```
openclaw config set provider deepseek
```

```
openclaw config set model deepseek/deepseek-v4
```

```
openclaw model test
```

6. Option E — OpenRouter (API key, routes to Claude Opus 4.7). Get a key from <https://openrouter.ai/keys>:

```
export OPENROUTER_API_KEY="sk-or-..."
```

```
openclaw config set provider openrouter
```

```
openclaw config set model openrouter/anthropic/claude-opus-4.7
```

```
openclaw model test
```

7. Persist API keys so they survive reboots. For the API-key providers, add the export line to your shell profile (~/.zshrc on macOS, ~/.bashrc on Linux). OAuth tokens are persisted automatically in ~/.openclaw/auth/ and do not need this.

```
echo 'export ANTHROPIC_API_KEY="sk-ant-..."' >> ~/.zshrc
```

```
source ~/.zshrc
```

8. Switch models and confirm the active one. You can hot-swap the global default at any time:

```
openclaw model use deepseek/deepseek-v4
```

```
openclaw model current
```

```
openclaw model test
```

9. Give the agent a Nimbus Supplies smoke test. With any model active, from the CLI chat (openclaw) ask: > You are the back-office assistant for Nimbus Supplies, a small office-supplies reseller. In two sentences, introduce yourself to a customer. A sensible reply confirms the model is wired up.

Test it

openclaw models list shows the connected models, openclaw model current prints the active one, openclaw model test returns a short successful completion with no auth error, and OAuth credentials appear under ~/.openclaw/auth/.

Note: Full commands and screenshots are in labs/lab-16-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 17 — Channels

Learning outcome: LO2: Connect multiple messaging channels to one agent gateway..

Goal: The learner gives customers a way to reach the Nimbus Supplies agent by creating a Telegram bot end-to-end via BotFather, registering it with a token, pairing a WhatsApp number by scanning a QR code, and running both channels at once through a single OpenClaw gateway so the same agent answers on either platform.

What you'll build

Telegram and WhatsApp both live on one OpenClaw gateway, answered by the same Nimbus Supplies agent (Tools: OpenClaw, Telegram (BotFather), WhatsApp, openclaw channel.)

Step-by-step

1. Create a Telegram bot with BotFather. 1. Open Telegram and search for @BotFather. 2. Send /newbot. 3. Enter a display name (e.g. Nimbus Supplies Assistant). 4. Enter a username — must be unique and end in _bot (e.g. nimbus_supplies_bot). 5. BotFather replies with an HTTP API token like 123456789:ABC-DEF1234ghIkl-zyx57W2v1u123ew11. Copy it.
2. Register and start the Telegram channel.

```
openclaw channel add telegram --token 123456789:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
```

```
openclaw channel start telegram
```

3. Test Telegram. In Telegram, search for your bot's username and send: > Hi, do you supply recycled A4 paper for Nimbus Supplies? The agent should reply using the model you connected in Lab 16.
4. Add and start the WhatsApp channel.

```
openclaw channel add whatsapp
```

```
openclaw channel start whatsapp
```

5. Pair WhatsApp by scanning the QR. On your phone: WhatsApp → Settings → Linked Devices → Link a Device, then scan the QR code shown in the terminal. Wait for whatsapp: connected in the OpenClaw log. > WhatsApp keeps a stateful session

on disk at ~/.openclaw/whatsapp/. Do not delete that folder unless you intend to re-pair.

6. Confirm both channels are running at the same time. One gateway hosts many channels — the same Nimbus Supplies agent now answers on both.

```
openclaw channel list

   openclaw gateway status
```

7. Inspect and manage a channel. Useful day-to-day commands:

```
openclaw channel show telegram

   openclaw channel restart whatsapp   # e.g. after the QR expires
```

Test it

openclaw channel list shows telegram and whatsapp both running, a message to the Telegram bot gets an agent reply, a message to the linked WhatsApp account gets an agent reply, and openclaw gateway status shows both connected.

Note: Full commands and screenshots are in labs/lab-17-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 18 — Skills

Learning outcome: LO3: Extend an agent with registry and custom skills..

Goal: The learner installs and removes skills from the skills.sh registry, invokes them from chat, and builds a custom Nimbus Supplies skill (nimbus-quote) under ~/.openclaw/skills/ so the agent handles a recurring back-office task — drafting an itemised customer quote with GST — the same way every time.

What you'll build

Registry skills plus a custom nimbus-quote skill that produces itemised, GST-inclusive quotes (Tools: OpenClaw, skills.sh registry, ~/.openclaw/skills/, openclaw skills.)

Step-by-step

1. List the skills currently installed.

```
openclaw skills list
```

2. Install a few useful skills from the registry. Skills come from <https://skills.sh/>:

```
openclaw skills add web-research --source skills.sh

   openclaw skills add self-improvement --source skills.sh
```

3. Invoke a skill from chat. In your Telegram or WhatsApp channel, send:

```
/skill web-research "Who are the main office-supplies wholesalers in
Singapore?"
```

4. Remove a skill you do not need (keeps the agent focused and safe):

```
openclaw skills remove self-improvement
```

5. Refresh the registry index if a skill you expect is missing:

```
openclaw skills update
```

6. Build a custom Nimbus Supplies skill. Skills live under ~/.openclaw/skills/. Create a folder and a skill definition. (File layout is shown here as a documented example — confirm the exact schema for your version at <https://docs.openclaw.ai/> before relying on advanced fields.)

```
mkdir -p ~/.openclaw/skills/nimbus-quote
```

7. Register and test the custom skill.

```
openclaw skills list                # confirm nimbus-quote appears

   openclaw gateway restart            # reload skills if it is not picked
up
```

Test it

openclaw skills list shows the registry skills plus the custom nimbus-quote, /skill web-research runs end-to-end, and /skill nimbus-quote produces an itemised quote with a 9% GST line and standard terms, marking any unknown price as TBC instead of guessing.

Note: Full commands and screenshots are in labs/lab-18-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 19 — Tools & Integrations

Learning outcome: LO3: Extend an agent with third-party tool integrations..

Goal: The learner extends the Nimbus Supplies agent with two real integrations — Firecrawl (JS-rendered web scraping) and AgentMail (its own inbox/outbox) — then runs an end-to-end back-office task: research a supplier's website prices, draft an itemised quote with the Lab 18 skill, and email it to a customer. Tool profiles and allow/deny lists scope tools per channel.

What you'll build

A Firecrawl + AgentMail integrated agent that scrapes supplier prices and emails an itemised quote (Tools: OpenClaw, Firecrawl, AgentMail, openclaw tools.)

Step-by-step

1. Inspect the built-in tools first.

```
openclaw tools list
```

2. Enable Firecrawl. Sign up at <https://www.firecrawl.dev/>, then Dashboard → API Keys → copy the fc-... key:

```
export FIRECRAWL_API_KEY="fc-..."

openclaw tools enable firecrawl --api-key "$FIRECRAWL_API_KEY"

openclaw tools status firecrawl
```

3. Smoke-test Firecrawl from chat. In Telegram or WhatsApp: > Use Firecrawl to fetch the products page of a stationery supplier's website and list their A4 paper options with prices in 5 bullets.

4. Enable AgentMail. Create an inbox at <https://agentmail.to/>, copy the API key and inbox address:

```
export AGENTMAIL_API_KEY="..."

openclaw tools enable agentmail \

  --api-key "$AGENTMAIL_API_KEY" \

  --inbox bot@yourname.agentmail.to

openclaw tools status agentmail
```

5. Smoke-test AgentMail. From chat: > Send an email from my AgentMail inbox to <your-personal-email> with subject "Hello from Nimbus Supplies" and a friendly one-paragraph body. Check your inbox, reply to it, then ask: > Check my AgentMail inbox and summarise the latest reply.

6. Run the real end-to-end task (research → quote → email). From a channel, send one instruction that chains both tools plus the skill from Lab 18: > A customer, Acme Cafe, wants 10 reams of recycled A4 paper and 5 boxes of black pens. Use Firecrawl to check current prices on our supplier's site, then use the nimbus-quote skill to draft an itemised quote, and email it from my AgentMail inbox to orders@acmecafe.example with subject "Your Nimbus Supplies quote".

7. Scope tools per channel with profiles and allow/deny lists. Public channels should not run shell exec. Apply the safe messaging preset and lock down a public channel:

```
openclaw channel set telegram --profile messaging

openclaw channel set telegram --deny exec

openclaw channel set telegram --allow group:web

openclaw channel show telegram
```

8. (Optional) Add one more integration of your choice. Pick one from <https://docs.openclaw.ai/tools> and enable it (exact name/flags per the docs), e.g.:

```
openclaw tools enable <tool-name> --api-key <KEY>
```

Test it

openclaw tools list --enabled shows firecrawl and agentmail, the agent scrapes a supplier URL and returns structured prices, an AgentMail send arrives in a real inbox, the end-to-end task produces an itemised quote email, and a channel set to --deny exec refuses shell-exec requests.

Note: Full commands and screenshots are in labs/lab-19-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 20 — Commands

Learning outcome: LO4: Operate an agent through its CLI and in-chat command surface..

Goal: The learner becomes fluent in the two ways to drive OpenClaw daily: the CLI commands (config, model, doctor, gateway, channel, tools, skills, cron) and the in-chat slash commands operators and customers use inside a channel, building a personal quick-reference table for running the Nimbus Supplies back office and streaming live gateway logs.

What you'll build

A personal quick-reference of the CLI and slash commands used to run Nimbus Supplies day to day (Tools: OpenClaw, openclaw CLI, in-chat slash commands.)

Step-by-step

1. openclaw config — read and change settings.

```
openclaw config list                          # show all settings

   openclaw config get model                     # get one setting

   openclaw config set model anthropic/claude-opus-4-7

   openclaw config edit                           # open the config file in
$EDITOR
```

2. openclaw model — manage the brain.

```
openclaw model list

   openclaw model current

   openclaw model use deepseek/deepseek-v4

   openclaw model test
```

3. openclaw doctor — health check (with auto-fix).

```
openclaw doctor
```

```
openclaw doctor --fix
```

4. openclaw gateway — control the daemon that runs everything.

```
openclaw gateway status

openclaw gateway start

openclaw gateway stop

openclaw gateway restart

openclaw gateway logs --tail 50

openclaw gateway install     # install daemon (LaunchAgent / systemd /
Task Scheduler)
```

5. openclaw channel — manage the front doors.

```
openclaw channel list

openclaw channel add telegram --token <TOKEN>

openclaw channel start whatsapp

openclaw channel show telegram

openclaw channel restart whatsapp

openclaw channel remove telegram
```

6. openclaw tools and openclaw skills — capabilities.

```
openclaw tools list

openclaw tools list --enabled

openclaw tools status agentmail


openclaw skills list

openclaw skills add web-research --source skills.sh

openclaw skills remove web-research
```

7. In-chat slash commands. These work inside any channel (Telegram, WhatsApp, CLI chat). Try each from your Nimbus Supplies bot:

8. Watch it work live. In one terminal, follow the logs; in the channel, send a message and observe the corresponding lines:

```
openclaw gateway logs --follow
```

Test it

You can switch the model both from chat (/model, per-conversation) and globally from the CLI (openclaw model use), /whoami returns your platform user ID, openclaw gateway logs --follow streams live activity, and /help lists the available slash commands.

Note: Full commands and screenshots are in labs/lab-20-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 21 — Cron Jobs & Heartbeat

Learning outcome: LO4: Automate agent work with scheduled crons and a monitoring heartbeat..

Goal: The learner makes Nimbus Supplies run unattended by scheduling recurring tasks with cron (a nightly sales report and a weekly supplier price-check using Firecrawl) and monitoring uptime with the heartbeat, so the gateway self-checks, auto-restarts on failure, and can page an external monitor if it dies.

What you'll build

A nightly sales-report cron, a weekly price-check cron, and an enabled self-healing heartbeat (Tools: OpenClaw, openclaw cron, gateway heartbeat, Firecrawl + AgentMail.)

Step-by-step

1. Create the nightly sales-report cron. Cron syntax is <min> <hour> <day> <month> <weekday>.

```
# Every day at 21:00 — post a Nimbus Supplies end-of-day summary to
Telegram

   openclaw cron create \

     --schedule "0 21 * * *" \

     --prompt "Summarise today's Nimbus Supplies customer chats and any
quotes sent. List open questions that still need a human." \

     --channel telegram \

     --name nightly-sales-report
```

2. Create a weekly supplier price-check cron (uses Firecrawl from Lab 19):

```
# Every Monday at 08:00 — check supplier prices and email me

   openclaw cron create \

     --schedule "0 8 * * 1" \
```

```
     --prompt "Use Firecrawl to fetch our main supplier's A4 paper price
and email me the current price via AgentMail with subject 'Weekly supplier
price'." \

     --channel telegram \

     --name weekly-price-check
```

3. List and inspect your crons.

```
openclaw cron list

   openclaw cron show nightly-sales-report

   openclaw cron logs nightly-sales-report --tail 20
```

4. Fire a cron on demand to test it now (ignores the schedule):

```
openclaw cron run nightly-sales-report
```

5. Disable / enable / delete crons as your needs change:

```
openclaw cron disable weekly-price-check

   openclaw cron enable  weekly-price-check

   openclaw cron delete  weekly-price-check   # only if you no longer need
it
```

6. Enable the heartbeat. The heartbeat is a periodic self-check that verifies the gateway is alive, the model responds, and channels are connected; on failure the daemon attempts auto-restart and logs the incident.

```
openclaw gateway heartbeat enable --interval 60s

   openclaw gateway heartbeat status

   openclaw gateway heartbeat logs --tail 20
```

7. (Optional) Push a heartbeat ping to an external monitor so you are paged if Nimbus Supplies goes dark (e.g. a free <https://healthchecks.io/> check):

```
openclaw gateway heartbeat webhook \

     --url https://hc-ping.com/<your-uuid> \

     --interval 5m
```

8. Confirm auto-restart works. Ensure the daemon is installed to auto-start, then simulate a crash:

```
openclaw gateway status        # expect "running" + "auto-start: yes"

   openclaw gateway stop

   # wait ~60s for the service manager to restart it
```

```
openclaw gateway status        # expect "running" again
```

Test it

openclaw cron list shows nightly-sales-report and weekly-price-check, openclaw cron run delivers a summary to Telegram immediately, openclaw gateway heartbeat status reports healthy, and after openclaw gateway stop the service manager restarts the gateway within about a minute.

Note: Full commands and screenshots are in labs/lab-21-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 22 — Memory & Context

Learning outcome: LO2: Give an agent persistent, user-scoped memory..

Goal: The learner gives the Nimbus Supplies agent durable memory so it recalls customer preferences across conversations: using the in-chat /memory command, inspecting where OpenClaw persists state under ~/.openclaw/, teaching a customer's standing preference, clearing the conversation to prove it is real memory, and confirming recall in a later separate chat.

What you'll build

An agent that persistently remembers a customer's standing preferences and applies them in a fresh conversation (Tools: OpenClaw, /memory, /clear, ~/.openclaw/ state.)

Step-by-step

1. See what OpenClaw stores on disk.

```
ls -la ~/.openclaw/
```

2. Inspect your current memory from chat. In your Telegram or WhatsApp bot:

```
/memory
```

3. Teach the agent a durable customer preference. Still in chat, send a plain-language instruction: > Remember this for future chats: our customer Acme Cafe always wants recycled A4 paper (never standard), delivery on Tuesdays only, and quotes addressed to Mei from Acme. Then confirm it was captured:

```
/memory
```

4. Clear the conversation to prove it is real memory, not just context.

```
/clear
```

5. Recall in a fresh conversation. In the now-cleared chat, ask something that requires the remembered facts: > Draft a quote for Acme Cafe: 10 reams of A4 paper and 5 boxes of black pens. A correct agent should automatically choose

recycled A4, note Tuesday delivery, and address the quote to Mei — without you repeating any of it. (This pairs perfectly with the nimbus-quote skill from Lab 18.)

6. Edit or correct a memory. If a preference changes, update it in natural language and re-check: > Update your memory: Acme Cafe now accepts delivery on Tuesdays or Thursdays.

```
/memory
```

7. Back up memory before major changes (memory lives under ~/.openclaw/, so the Lab 15 backup habit applies):

```
mkdir -p ~/openclaw-backup-$(date +%F)

   cp -R ~/.openclaw/ ~/openclaw-backup-$(date +%F)/openclaw-state
2>/dev/null || true
```

Test it

/memory shows the customer preferences after you teach them, after /clear the /memory list still shows them (short-term context cleared, long-term memory retained), and a fresh quote request for the customer automatically applies recycled paper, Tuesday delivery and the correct recipient name without repetition.

Note: Full commands and screenshots are in labs/lab-22-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 23 — Security

Learning outcome: LO4: Secure an agent with key management, allowlists, sandboxing and audit..

Goal: The learner hardens the Nimbus Supplies deployment before real customers: storing API keys as environment variables (never in cleartext config), restricting who can DM the bot with allowlists, applying per-channel tool deny lists, tightening the code-execution sandbox (timeout/memory/network), reviewing the audit log, and rotating provider keys.

What you'll build

A hardened agent with keys in env, DM allowlists, per-channel deny lists, a capped sandbox and an exported audit log (Tools: OpenClaw, allowlists, tool deny lists, exec/python sandbox, gateway logs.)

Step-by-step

1. Secure API keys — never hardcode or commit them. Store keys once as environment variables. macOS / Linux — append to ~/.zshrc or ~/.bashrc:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

```
export FIRECRAWL_API_KEY="fc-..."

export AGENTMAIL_API_KEY="..."
```

2. Restrict who can DM the bot (allowlists). By default anyone who finds your Telegram bot can message it. Lock it to known users:

```
openclaw channel set telegram \

  --allow-users 12345678,87654321 \

  --pair-mode allowlist
```

3. Apply tool deny lists (least privilege per channel). A customer-facing channel has no business running shell commands or deleting files:

```
openclaw channel set telegram --deny exec,fs.write,fs.delete

openclaw channel set telegram --allow group:web

openclaw channel set telegram --profile messaging

openclaw channel show telegram | grep -E "(allow|deny|profile)"
```

4. Tighten the code-execution sandbox. The exec / Python tools run sandboxed — cap time, memory, and network:

```
openclaw tools config exec \

  --timeout 10s \

  --memory 256mb \

  --network deny


openclaw tools config python \

  --timeout 30s \

  --memory 512mb \

  --network deny
```

5. Review the audit log. Every tool call, model call, and channel event is logged:

```
openclaw gateway logs --tail 100

openclaw gateway logs --filter tool=exec

openclaw gateway logs --since 1h --filter level=warn

openclaw gateway logs --export ~/openclaw-audit-$(date +%F).log
```

6. Rotate provider keys on a schedule (e.g. every 90 days). 1. Generate a new key in the provider dashboard. 2. Update the environment variable. 3. Restart the

gateway: openclaw gateway restart. 4. Confirm openclaw model test still works. 5. Revoke the old key in the provider dashboard.

Test it

cat ~/.openclaw/config.toml does not contain raw API keys, a user not on the Telegram allowlist gets a not-authorized response, openclaw channel show telegram lists the deny list and messaging profile, and the audit log shows blocked exec attempts on a locked channel.

Note: Full commands and screenshots are in labs/lab-23-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 24 — Multi-Agent Workforce

Learning outcome: LO5: Orchestrate cooperating multi-agent teams to deliver a business outcome..

Goal: In the OpenClaw capstone the learner splits the single agent into three cooperating agents — Sales (customer-facing on Telegram/WhatsApp), Research (supplier scouting with Firecrawl), and Ops (email, quotes, nightly crons) — each with least-privilege tools, and runs one end-to-end business scenario across all three, with a verified fallback using isolated OPENCLAW_HOME instances that cooperate via AgentMail and a shared folder.

What you'll build

A three-agent Sales/Research/Ops workforce that runs a customer order end-to-end into a real quote email (Tools: OpenClaw, openclaw agent (or isolated OPENCLAW_HOME instances), Firecrawl, AgentMail.)

Step-by-step

1. Create three named agents. Attempt the native multi-agent path first (confirm exact syntax in the docs):

```
openclaw agent create sales

   openclaw agent create research

   openclaw agent create ops

   openclaw agent list
```

2. Give each agent least-privilege tools (mirrors Lab 19/23 allow-deny, applied per agent):

```
# Sales — web + quoting only, no shell, no filesystem writes

   openclaw agent set sales --allow group:web --deny
exec,fs.write,fs.delete
```

```
   # Research — scraping only

   openclaw agent set research --allow firecrawl,web_search,browser --deny
exec,fs.write



   # Ops — email + files, no public channel exposure

   openclaw agent set ops --allow agentmail,group:fs
```

3. Attach channels to the right agents.

```
# Customer traffic goes to Sales

   openclaw channel set telegram --agent sales

   openclaw channel set whatsapp --agent sales



   # A private research bot for the operator (see Lab 17 for creating a 2nd
bot)

   openclaw channel add telegram --token <RESEARCH_BOT_TOKEN>

   openclaw channel set <research-bot> --agent research
```

4. Give each agent the right skill/memory. Sales uses nimbus-quote and the customer memory from Lab 22; Research uses nimbus-supplier-brief (Lab 18 exercise). Install/enable as needed:

```
openclaw skills list
```

5. Wire cooperation between agents. The simplest, robust hand-off uses the tools you already have: - Sales → Ops: when Sales captures an order, it asks Ops (via a shared note file or an AgentMail message) to produce and send the quote. - Ops → Research: when a price is unknown ("TBC"), Ops asks Research to look it up with Firecrawl. - Research → Ops: Research replies with the price; Ops finalises the quote email.

```
# Ops nightly consolidation cron (from Lab 21, now owned by Ops)

   openclaw cron create \

     --schedule "0 21 * * *" \

     --prompt "Compile today's Nimbus Supplies orders and quotes sent, and
list anything waiting on Research." \

     --channel telegram \

     --name ops-nightly-report
```

6. Fallback (fully verified) — run agents as separate profiles/instances. If native multi-agent commands are not in your build, model each agent as its own OpenClaw configuration using only Lab 15–19 primitives: - Sales = your main instance with Telegram/WhatsApp, --profile messaging, deny exec,fs.write (Lab 23). - Research = a second instance pointed at a separate config dir with only Firecrawl/web tools enabled:

```
OPENCLAW_HOME=~/.openclaw-research openclaw onboard
```

```
  OPENCLAW_HOME=~/.openclaw-research openclaw tools enable firecrawl --
api-key "$FIRECRAWL_API_KEY"
```

7. Run the end-to-end capstone scenario. From a customer's Telegram, trigger the whole workflow: > Hi Nimbus Supplies — I'm Mei from Acme Cafe. I need 10 reams of recycled A4 paper and 5 boxes of black pens. What's your best price and when can you deliver? Expected chain: 1. Sales greets Mei, recalls Acme's preferences from memory (recycled paper, Tuesday delivery — Lab 22), and captures the order. 2. Sales hands the order to Ops, which starts the nimbus-quote. The pen price is unknown → marked TBC. 3. Ops asks Research to fetch the current pen price via Firecrawl. 4. Research replies with the price; Ops finalises the itemised quote (with GST) and emails it to Acme via AgentMail. 5. That night, the ops-nightly-report cron summarises the day, including Mei's order.

Test it

Three agents (or isolated instances) each have a distinct least-privilege tool set (Sales cannot exec, Research cannot write files, Ops is not on a public channel), a customer Telegram message produces a real quote email combining memory, the nimbus-quote skill, a Firecrawl price lookup and AgentMail delivery, and the nightly cron posts a summary referencing the same order.

Note: Full commands and screenshots are in labs/lab-24-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 25 — Dreaming — Idle Reflection & Self-Improvement

Learning outcome: LO3: Use OpenClaw 'dreaming' so the agent reflects during idle time..

Goal: Enable OpenClaw's 'dreaming' feature so the Nimbus Supplies agent uses idle time to review its memory, consolidate what it has learned, and generate useful follow-up actions and skill improvements — then inspect what it produced. Reference the OpenClaw functions playlist for the walkthrough.

What you'll build

An agent that, when idle, 'dreams' — reflecting on memory and proposing follow-ups you can review. (Tools: OpenClaw, dreaming, memory.)

Step-by-step

1. Enable the dreaming feature in OpenClaw config. This flips the setting that lets the gateway schedule reflection cycles when the agent is idle:

```
openclaw config set dreaming.enabled true
```

2. Let the agent sit idle so a dream cycle runs — or trigger one manually. Rather than wait for a natural idle window, force a cycle now so you can observe it immediately:

```
openclaw dream run
```

3. Review what the dream produced. List the insights, follow-ups, and skill suggestions the cycle generated:

```
openclaw dream list
```

4. Accept a useful follow-up the agent proposed. In the dashboard (or via the CLI shown in the video), open one proposal that genuinely helps Nimbus Supplies and approve it. Accepting promotes the proposal into a real action or a saved skill; declining discards it. This human review step keeps you in control of what the agent's downtime actually changes.

Test it

A dream cycle runs during idle time and produces reviewable insights/follow-ups from the agent's memory.

Note: Full commands and screenshots are in labs/lab-25-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 26 — Use Cases & Key Functions

Learning outcome: LO3: Apply OpenClaw's key functions to real back-office use cases..

Goal: Tie the OpenClaw track together with practical use cases for Nimbus Supplies — a Google Workspace CLI assistant, a data-analytics agent, and a social-media-marketing agent — combining channels, skills, tools, memory, dreaming and crons into end-to-end automations. Each use case has its own reference video.

What you'll build

One or more end-to-end OpenClaw automations (Workspace, analytics or social media) solving a real use case. (Tools: OpenClaw, skills, tools, Google Workspace, crons, memory.)

Step-by-step

1. Watch one of the use-case videos above and study how the presenter composes several functions into one automation — which channel triggers the flow, which skills and tools it calls, and how memory and crons fit in. Good starting points: a Google Workspace CLI assistant (Docs/Sheets/Gmail), a Data Analytics agent, or a Social Media Marketing agent.

2. Pick a back-office use case. Choose one concrete Nimbus Supplies workflow to implement end-to-end. A strong default is "research a supplier, then email a quote": a customer asks for a price, the agent looks up current supplier pricing, applies a margin, and emails a formatted quote. Other options: customer FAQ Q&A on Telegram, or an automated nightly orders-and-quotes report.

3. Compose the channels, skills and tools needed for it. Wire the pieces together for your chosen use case. For research-to-quote that means: the customer channel (e.g. Telegram) as the trigger, the skill that formats a quote (e.g. nimbus-quote from Lab 18), a research tool (Firecrawl / web_search) for live pricing, and a delivery tool (AgentMail) to send the email. Confirm the agent has permission to call each one.

4. Run the automation end-to-end and verify the outcome. Send a real trigger — e.g. message the agent "What's your price for 50 kg of arabica beans?" on the customer channel — and watch the full chain fire: the agent recalls context from memory, runs the price lookup tool, builds the quote with the skill, and delivers it via email. Confirm the actual deliverable (the sent quote email) is correct, not just that the logs look busy.

5. Schedule it or hand it to the multi-agent team from Lab 24. Make the automation ongoing. Either attach it to a cron (e.g. a nightly consolidation report) so it runs unattended, or delegate the use case to the Sales / Research / Ops agents you built in Lab 24 so the right specialist owns each step. This is what turns a one-off demo into part of an always-on Nimbus Supplies back office.

Test it

A real use case (Google Workspace, data analytics or social media) runs end-to-end using OpenClaw's channels, skills, tools and crons.

Note: Full commands and screenshots are in labs/lab-26-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Topic 03 — Paperclip — Running & Governing a Company of AI Agents (34%)

Setup · Connect OpenAI Codex · Configure · Track Tasks · Automate Hiring · Setup Agents · Security · Assign Tasks

Key concepts

- Paperclip is an operating system for a company of AI agents, self-hosted with Docker Compose; you are the Board, a CEO agent reports to you, and specialists report to the CEO.
- You connect a coding model such as OpenAI Codex as the agents' engine, configure the company, and track AI tasks on a board: todo -> in_progress -> in_review -> done.
- Hiring is automated behind human approval gates; all durable state lives in an embedded PostgreSQL database whose activity_log (tagged by actor_type) and cost_events tables give a CFO-style audit trail.
- Security is enforced with budget caps (an 80% warning and a 100% pause rail) and approvals; you set up agents and assign tasks to deliver the company goal.

Lab 27 — Setup Paperclip

Learning outcome: LO1: Install and self-host Paperclip, then found your company..

Goal: Watch the Paperclip overview, then self-host Paperclip with Docker Compose and create the Nimbus Coffee company with a single goal and a monthly budget.

What you'll build

A running Paperclip at http://localhost:3100 with a company (goal + budget). (Tools: Paperclip, Docker Compose.)

Step-by-step

1. Clone the Paperclip repository. This pulls the source and the Docker Compose files you need to run it locally:

```
git clone https://github.com/paperclipai/paperclip.git
```

```
cd paperclip
```

2. Start Paperclip with the quickstart compose file. This builds the image and brings up the full stack (the app plus its embedded PostgreSQL):

```
docker compose -f docker-compose.quickstart.yml up --build
```

3. Open the dashboard. In your browser, go to:

4. Create the company with a single goal and a monthly budget. In the dashboard, start a new company. Name it Nimbus Coffee, Inc., give it one clear goal (e.g. "Launch a direct-to-consumer specialty coffee brand and make the first sale"), and set a monthly budget cap (your spend ceiling). Save. This company is the entity the CEO and specialist agents will work inside for every remaining Paperclip lab. > The exact wording of the "Create company" / goal / budget fields is version-specific — verify in the video / docs (<https://docs.paperclip.ing>).

Test it

The dashboard loads at http://localhost:3100 and a company exists with a goal and budget.

Note: Full commands and screenshots are in labs/lab-27-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 28 — Connect OpenAI Codex to Paperclip

Learning outcome: LO2: Connect a coding model (OpenAI Codex) as the agents' engine..

Goal: Give Paperclip's agents a brain by connecting the OpenAI Codex adapter, so the agents can reason and act. Confirm the adapter is detected and available.

What you'll build

Paperclip using the OpenAI Codex adapter as the agents' model. (Tools: Paperclip, OpenAI Codex.)

Step-by-step

1. Install / log in to the OpenAI Codex CLI so Paperclip can detect it. Follow the video's instructions to install the Codex CLI and authenticate (log in or supply your API key). Paperclip detects the adapter by finding this working Codex CLI/credentials on the host — so getting the login right here is the whole ballgame. > The exact install/login command is version-specific — verify in the video / docs (<https://docs.paperclip.ing>).
2. Open Settings → Agents / Adapters in the dashboard. In Paperclip at http://localhost:3100, go to Settings, then the Agents / Adapters section. This is where Paperclip lists every model engine it can use to power agents.
3. Confirm the OpenAI Codex adapter shows as "available". Look for the OpenAI Codex entry and check its status reads available (not "not detected" / "unavailable"). Available means Paperclip successfully found and authenticated the Codex CLI and can route agent reasoning through it.
4. Restart Paperclip if the adapter is not detected. If Codex still shows unavailable after you have logged in, restart the stack so Paperclip re-scans for the adapter:

```
docker compose -f docker-compose.quickstart.yml restart
```

Test it

The OpenAI Codex adapter is detected and shows as available in Settings.

Note: Full commands and screenshots are in labs/lab-28-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 29 — Configure Paperclip

Learning outcome: LO2: Configure the company — settings, budget and workspace..

Goal: Configure Nimbus Coffee: set the monthly budget, connect a workspace folder so agents write real files, and adjust company settings.

What you'll build

A configured company with a budget and a connected workspace folder. (Tools: Paperclip, workspace.)

Step-by-step

1. Open the company's settings in the dashboard. At http://localhost:3100, open Nimbus Coffee, Inc. and go to its Settings. This is the per-company control panel (distinct from the app-wide Settings you used for adapters in Lab 28).
2. Set or adjust the monthly budget cap. Enter the maximum the company may spend per month. This cap is the governance rail you will exercise in Lab 33 (an 80% warning and a 100% pause). Set a deliberate figure you are comfortable letting agents spend against. > Field name/units are version-specific — verify in the video / docs (<https://docs.paperclip.ing>).
3. Connect a workspace folder so agents create real deliverables. First create the folder on your machine:

```
mkdir -p ~/paperclip-workspace/nimbus-coffee
```

4. Save the configuration and confirm it persists. Save the settings, reload the page, and confirm the budget and the connected workspace path are still shown. Persistence matters — it proves the config was written to the database, not just held in the browser.

Test it

The company shows the configured budget and a connected workspace folder.

Note: Full commands and screenshots are in labs/lab-29-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 30 — Track AI Tasks

Learning outcome: LO3: Track AI work on the task board..

Goal: Use Paperclip's task board to track work through its four states — todo, in_progress, in_review, done — giving you visibility and control over what the agents are doing.

What you'll build

A task board showing AI tasks moving across the four states. (Tools: Paperclip, task board.)

Step-by-step

1. Open the task board for the company. In the dashboard, open Nimbus Coffee, Inc. and go to its task board. You'll see four columns — todo, in_progress, in_review, done — the single source of truth for everything the company is working on.
2. Create or observe a task in the "todo" column. Either create a task yourself (e.g. "Draft the Nimbus Coffee brand tagline") or watch one the CEO/agents have already queued. A task in todo is committed work that has not started yet.
3. Watch a task move todo → in_progress → in_review. As an agent picks up the task, it moves to in_progress; when the agent finishes and produces a deliverable, it moves to in_review, waiting for your judgement. Observing this flow is how you supervise a company of agents without micromanaging each action.
4. Open a shell to review the task activity log. For a deeper view than the UI, open a shell inside the running Paperclip container:

```
docker compose -f docker-compose.quickstart.yml exec paperclip sh
```

Test it

Tasks are visible on the board and move through todo -> in_progress -> in_review -> done.

Note: Full commands and screenshots are in labs/lab-30-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 31 — Automate AI Hiring

Learning outcome: LO3: Automate hiring of the CEO and specialist agents behind approval gates..

Goal: Hire your CEO agent and let it propose specialist hires, each passing a human approval gate. This is how the company staffs itself automatically while you stay in control.

What you'll build

An approved CEO agent and one or more approved specialist agents. (Tools: Paperclip, approval gates.)

Step-by-step

1. Hire the CEO agent from the company page. On the Nimbus Coffee, Inc. page, choose to hire the CEO. The CEO is the top agent that reads the company goal and works out what roles the company needs to achieve it.

2. Approve the CEO hire (you are the Board). The hire pauses at an approval gate. Review it and approve — as the Board, nothing joins the company without your sign-off. This is the core governance pattern: agents propose, humans approve.

3. Let the CEO propose specialist hires. Once active, the CEO analyses the goal and proposes specialists it needs (for a coffee brand, perhaps a Marketer, an Engineer for the storefront, and an Operations agent). Each proposal explains the role and why it helps reach the goal.

4. Review and approve each specialist at the approval gate. For every proposed hire, review the role and approve or reject it at the gate. Approve the ones that genuinely serve the Nimbus Coffee goal; reject any that don't. Each approval is recorded as a human decision — an actor_type=user entry in the audit trail you'll inspect in Lab 33.

Test it

The CEO and approved specialists are active; each hire is logged as an actor_type=user decision.

Note: Full commands and screenshots are in labs/lab-31-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 32 — Setup AI Agents

Learning outcome: LO2: Set up and configure the AI agents and their capabilities..

Goal: Configure the hired agents — their mandate, tools and per-agent budget caps — so each specialist can carry out the delegated work for Nimbus Coffee.

What you'll build

Configured agents with mandates, tools and per-agent budgets. (Tools: Paperclip, agents.)

Step-by-step

1. Open an agent's configuration. On the Nimbus Coffee page, pick one specialist (e.g. the Marketer) and open its configuration. This is where you turn a generic hire into a purpose-built role.

2. Set the agent's mandate and the tools it may use. Write a crisp mandate — one or two sentences on exactly what this agent is responsible for (e.g. "Own Nimbus Coffee's brand voice and launch marketing copy"). Then grant only the tools it needs (e.g. file-write to the workspace, web research) and withhold the rest. Least privilege here prevents an agent from straying outside its lane. > Tool names and the mandate field are version-specific — verify in the video / docs (<https://docs.paperclip.ing>).

3. Set a per-agent budget cap. Give this agent its own spend ceiling, carved out of the company budget. Per-agent caps mean one runaway agent can't drain the whole company — a key safety rail you'll exercise in Lab 33.

4. Save and confirm the agent is ready to receive tasks. Save the configuration, reload, and confirm the mandate, tools, and budget persist and the agent's status shows it is ready for work. Repeat steps 2–5 for each specialist you approved.

Test it

Each agent has a mandate, allowed tools and a budget cap, and is ready for tasks.

Note: Full commands and screenshots are in labs/lab-32-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 33 — Security

Learning outcome: LO3: Secure the company with budgets, safety rails and approvals..

Goal: Apply Paperclip's governance: company-wide and per-agent budget caps trigger an 80% warning and a 100% pause; approvals gate risky decisions; the audit trail records everything.

What you'll build

Budget caps with the 80% warning and 100% pause rails demonstrated, plus an audit review. (Tools: Paperclip, budgets, PostgreSQL audit.)

Step-by-step

1. Set a company and/or per-agent budget cap. In the company settings (Lab 29) and/or an agent's config (Lab 32), set a deliberately small cap so ongoing agent work will approach it during this lab. This is what lets you observe the safety rails fire instead of waiting days.

2. Trigger the 80% warning and 100% pause, then resume. Let the agents keep working (assign a task or two). As spend crosses 80% of the cap, Paperclip raises a warning; at 100% it pauses the company/agent so nothing can overspend. Review the pause, then resume (e.g. by raising the cap or explicitly resuming) and confirm work continues. You have now seen both the soft and hard money rails in action.

3. Audit actions and spend in the embedded PostgreSQL. Connect to Paperclip's database to inspect the ground-truth record:

```
psql "postgresql://postgres@localhost:5432/paperclip"
```

4. Separate human vs agent actions with actor_type. Run the audit query to see how much of the company's activity was human decisions versus agent actions:

```
SELECT actor_type, COUNT(*) FROM activity_log GROUP BY actor_type;
```

Test it

The 80% warning and 100% pause fire, resume works, and the audit shows actions by actor_type and total spend.

Note: Full commands and screenshots are in labs/lab-33-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Lab 34 — Assign Task

Learning outcome: LO3: Assign and delegate tasks to run the company end-to-end..

Goal: Delegate real work: assign tasks to specialists, watch them execute and produce deliverables in the workspace, and review and accept the results to complete the Nimbus Coffee goal.

What you'll build

Assigned tasks executed by agents, producing real deliverables you review and accept. (Tools: Paperclip, task board, workspace.)

Step-by-step

1. Assign a task to a specialist agent. On the task board (Lab 30), create a concrete task and assign it to the right specialist — e.g. give the Engineer "Build a one-page Nimbus Coffee landing page in the workspace", or the Marketer "Write the launch tagline and hero copy". Match the task to the mandate you set in Lab 32.
2. Watch the agent execute and write output to the workspace. The task moves todo → in_progress as the agent works. When it finishes, check that a real file appeared in your workspace:

```
ls -R ~/paperclip-workspace/nimbus-coffee
```

3. Review the deliverable in the in_review state. The task is now in in_review, waiting on you. Open the produced file and judge it against the task and the company goal. As the Board, you are the quality gate.
4. Approve to move it to done, or send it back for revision. If the deliverable is good, approve it — the task moves to done and that slice of the Nimbus Coffee goal is complete. If it falls short, send it back for revision with feedback, and the agent iterates. Either way, you stayed in control end-to-end.

Test it

An assigned task is executed by an agent, produces a real file, and is reviewed and accepted to done.

Note: Full commands and screenshots are in labs/lab-34-*.md. Use only accounts, keys and hosts you own, and keep agents under human oversight.

Wrap-Up — The Agent Build Arc and Governance

These cross-cutting themes run through every platform in the course. Study this section alongside the labs so the same skillset transfers from Hermes to OpenClaw to Paperclip and into your own production agents.

The agent build arc

Every platform follows the same progression — the order in which you built each agent in the labs:

- Install — set up the runtime (Hermes CLI/TUI, the OpenClaw Node.js gateway daemon, or Paperclip via Docker Compose).
- Models — connect an LLM provider (Claude, OpenAI, OpenRouter, MiniMax or DeepSeek) and pick a capable model.
- Channels — reach the agent where work happens (Telegram, WhatsApp, Discord, Slack and more).
- Memory — give the agent durable, user- or company-scoped recall across sessions and surfaces.
- Skills & tools — extend the agent with reusable skills and third-party tool/integration calls.
- Crons & heartbeat — schedule recurring work and let a heartbeat keep the agent running.
- Security — isolate execution in sandboxes, store secrets in config, and gate risky actions with approvals.
- Multi-agent — orchestrate a coordinator plus specialist workers to deliver an end-to-end business outcome.

Human-in-the-loop governance

- Approvals — important or destructive actions pause for explicit human sign-off before they run.
- Budgets — company- and per-agent spend caps warn at 80% and pause at 100% so costs never run away.
- Sandboxing — agents execute in isolated backends (Docker, SSH, or a remote host) so the host stays safe.
- Audit — an activity log and cost-events trail record who did what, giving you a CFO-style view of the workforce.
- Least privilege — scope each channel, skill and tool with allow/deny lists so agents can only do their job.

Next Steps

- First pass: complete every lab on your own machine, following the commands in each lab file.
- Second pass: redo the labs from memory until installing, connecting and securing an agent is automatic.
- Extend the labs with your own skills, tools and integrations for a workflow you care about.
- Deploy an agent on a VPS or cloud backend for 24/7 operation, with budgets, approvals and audit turned on.
- Explore multi-agent orchestration — a coordinator plus specialists — for a realistic end-to-end business task.
- Review each lab's detailed steps in this guide and re-run the labs on your own machine.

Glossary

- **Autonomous AI agent —** An LLM-driven system that perceives, reasons, acts with tools and remembers, running under human oversight.
- **Agent stack —** The layers that make an agent work: model, memory, skills, tools, channels and a scheduler/security layer.
- **Model / provider —** The LLM (and the vendor serving it) that gives the agent its reasoning — e.g. Claude, OpenAI, OpenRouter.
- **Channel —** A messaging surface the agent is reachable on, such as Telegram, WhatsApp, Discord or Slack.
- **Memory —** Durable, user- or company-scoped storage that lets the agent recall context across sessions and surfaces.
- **Skill —** A reusable, named capability an agent can install or create and invoke on demand.
- **Tool / integration —** An external action or service the agent can call — web search, image generation, email, GitHub via MCP.
- **MCP —** Model Context Protocol — a standard way to connect an agent to external tool servers.
- **Cron / heartbeat —** A schedule that triggers recurring agent work; a heartbeat self-checks and keeps the agent running.
- **Sandbox —** An isolated execution backend (Docker, SSH, remote host) that contains what an agent's commands can touch.
- **Approval gate —** A human sign-off required before an agent performs an important or destructive action.
- **Budget cap —** A spend limit at company or per-agent level that warns at 80% and pauses work at 100%.
- **Multi-agent orchestration —** A coordinator agent delegating to specialist worker agents to deliver an end-to-end outcome.